Automated test generation for `ctsa`

@hgfernan

February 27, 2025

Contents

Abstract

A routine for the automated test generation for the statistical library `ctsa` is outlined. It envolves the generation of simple tasks of model fitting and prediction using `ctsa`, compared with equivalent code in the Python libraries `pmdarima` and `statsmodels`, and in the R library `forecast`.

1 Motivation

Why using a test database and automatic generation of tests, instead of the handcrafted tests ?

2 Introduction

Overview of the project

3 Tables

Since the automated test generation is based on a database, here follows a description of the database to be used.

It has a mixed relational and document architecture: the main tables follow a conventional relational structure, but the parameters and test results are stored as JSON values. That's for pragmatic reasons: the parameters and test results are varied and have different structures. That could be easily mapped to a relational database structure, but it would be too laborious and cumbersome.

For instance: an $ARIMA$ model will have three basic parameters, while a $SARIMA$ model will have 7 basic parameters.

As such, test parameters and results will be stored as JSON objects encoded as strings, and dealt with by classes specialized in their content. There will be a class for each one of the models, that will be able to unpack and allow the use of the information in those JSON values.

A short description of the data base structure displayed in Figure 1 can be:

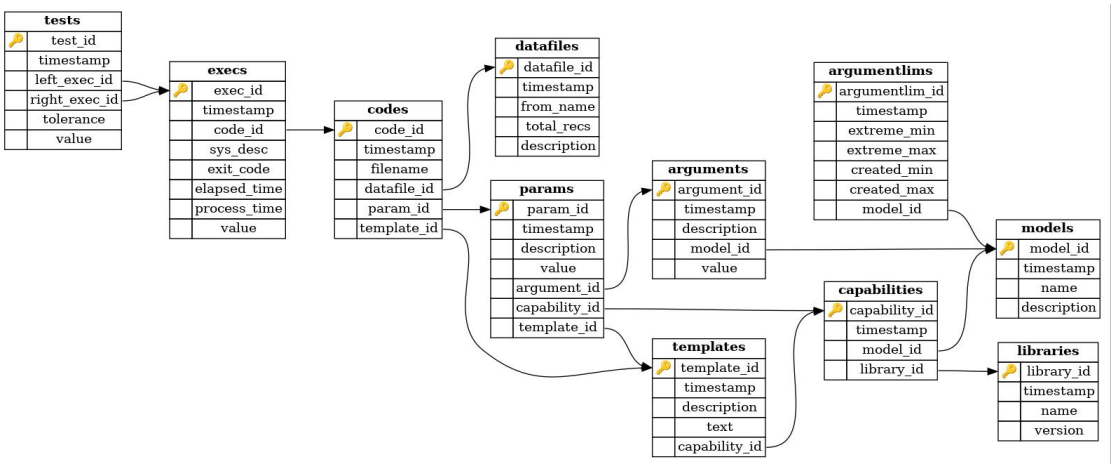

Figure 1: Database used for automated test generation

1. The most central table is `models` that contains the name of all statistical models in use.

   It is related to 3 other tables: `arguments` (item 5), `capabilities` (item 3), and `paramlims` (item 9);

2. The second central table is `libraries`. It contains the names of the libraries that shall be used in the tests.

   All libraries in use – that is `ctsa`, `forecast`, `pmdarima`, and `statsmodels` – are available in just one language, be it C, Python or R. Therefore there's currently no need to add a language field in this table.

   Another possible lack of information will be the partial support of a statistical model in one or more of the libraries, but for now its solution will be postponed;

3. Another important table is `capabilities`, that's a relationship between `models` (item 1) and `libraries` (item 2); that is: it informs what statistical models are supported by each library.

   This table guides the creation of code `templates` (item 7) for each language and library, as well as the adaptation of statistical `arguments` (item 5) for each library, in the form of `parameters` (item 6);

4. The table `datafiles` contains a list of data files specially prepared for the tests: they contain only a time series, in the fields `index` and `value`, to ease the creation of tests. As such they don't have a specific name: they're named with the `datafile_id` and the extension "`.csv`". For instance, the 1st file registered in this table will be named "`0001.csv`".

   The table field `from_name` relates this adapted data file to the original data where the data was obtained;

5. The table `arguments` contains values of elements of the statistical `models` (item 1), without any concern to its implementation in the `libraries` (item 2);

6. The table `parameters` contains the adaptation of the table `arguments` (item 5) to each implementation of one of the `models`) (item 1) of the codes of the `libraries` (item 2).

   That adaptation is done via the field `value`.

   This table refers the `capabilities` (item 3) to

   It is used to create a relate `models` and `libraries`;

7. The table `templates` contains codes templates to each one of the library `capabilities` (item 3) – that is all the statistical `models` (item 1) that all `libraries` (item 2) implement.

   A template is conceptually an open structure with holes where the `parameters` (item 6) and the `datafiles` (item 4) will be filled in. A template is also a fragment of source code where just these

two informations are missing;

8. The table `codes` contains the source codes generated by filling in each one of the `templates` (item 7) with one of `datafiles` (item 4) and one of `parameters` (item 6).

   The table `codes` a tiny of the table `templates`, but it's an important one, since makes more concrete this filling of a template.

   Another point to make matters more clear is that the name of a code file (contained in the field `filename`) won't be simply the transcription of the field `code_id` and an extension, but will refer to the field `name` of the corresponding row in `model` (item 1), of the `name` of used row in `libraries`, the `parameter_id` of the parameter used, and the `datafile_id` of the data file used in it.

   That is: the simple naming used in `datafiles` (item 4) won't be used here.

   For instance: if a `codes` row with `code_id` of 15, uses a C code template of library `ctsa`, its source code filename would be "`0015.c`". But there's a lot of parameters to consider when dealing with source code files, so another naming scheme will be used.

   If the code uses the *AR* model implementation of `ctsa`, uses the 1st group of parameters, and the `datafile_id` of 23, the extended name of its source code will be "`ctsa_ar_p0001_d0023.c`".

   In a typical Linux setup the executable name will be "`ctsa_ar_p0001_d0023`", and in a Microsoft Windows setup it will be "`ctsa_ar_p0001_d0023.exe`". However, the Windows version of the software will be postponed for a while.

   The library name will precede the model name because there's much more commonality between the creation of executable files of a library than there are between different implementations of the same statistical model in several different libraries.

9. The table `paramlims` contains the valid range of each parameter of a given model from the table `models` (item 1), from the value in the field `extreme_min` to the value `extreme_max`, inclusive.

   The execution of all codes – as contained in the table `execs` (item 10) – can take some time; therefore the parameters really used to create code files for all libraries will be kept in the closed interval of the field pair [`created_min`, `created_max`];

10. The table `execs` contains a history of the executions of a given code. Its only link is with the table `codes` (item 8).

    The fields in `execs` document the execution of a given code, identified by its `code_id`. The field `sys_desc` should contain the description of the machine where code was run, `exit_code` is the value returned by the execution of the code. The time of execution is measured by `elapsed_time` (the difference between the time the execution was started and the time it was finished), and `process_time` is the time of the CPU that was effetively used.

    The field `value` contains both the parameters used in to start the program, and the result it obtained. It contains the information presented as the summary of the fitting of the model to the data, and the result of the forecasting of a few time steps, both as the value obtained, and its deviation.

    That information is formatted as a JSON encoded string, due to the variety of parameters of each statistical model, as well as the variety of their measures of fitting.

11. Finally, the table `tests` is a comparison between two different executions of the same data, aka `datafile_id`, the same statistical model and the same model arguments; that is the same `argument_id`. One is called `lef_exec_id` and the other is `right_exec_id`.

    A test allows two different versions of the same library to be compared for the same model and arguments. Or two different libraries with the same model and arguments.

    The comparison can use the statistical or the results of the physical execution, like the process time or the exit code.

4 Software design

The operation of the software has the following steps:

1. **Data transcription**: The data series files available are adapted to a simple format CSV format that contains only an index column, and a value column.

   The table `file_db.csv` contains a mapping to the original data and information on the column that was used. For original data files with several value columns, several files in this simple format can be generated.

   This phase is partly manual, in the configuration of the `file_db.csv` file for each original datafile, and partly automatic, since the program `cvt_test.py` will convert automatically the data files to this test format, provided they're configured in `file_db.csv`

2. **Template creation**: This step is totally manual, and contains three substeps:

   (a) The existing `ctsa` test files are used to create `C` source code templates as Python strings that will have the parameter information filled in.

   From the `ctsa` source code templates it is extracted the statistical parameters that are named *arguments* in this context: they are the theoretical and universal part of the configuration used to be run the `ctsa` code. The pratical configuration, that includes parts specific to `ctsa` code is named *paramters*;

   (b) One or more Python libraries are identified with exactly the one or more algorithms that are the same as the ones in `ctsa`.

   The chosen library or libraries should accept the *arguments* (see the item 2a), but there's liberty to adjust the *arguments* to the library using a group of *parameters* specific to it;

   Currently, only `statsmodels` is being considered for generic algorithms, and `pmdarima` for automated hyperparameter tuning. But other libraries are available. In any case, the data structure defines templates by library, not by language. Therefore, any number of libraries with different languages can be used, provided they have the same algorithm as `ctsa`.

   In this substep, study and manual programming will lead to templates and parameters that generate results equivalent to the ones of `ctsa`;

   (c) One or more R libraries are identified with exactly the one or more algorithms that are the same as the ones in `ctsa`.

   The chosen library or libraries should accept the *arguments* (see the item 2a), but there's liberty to adjust the *arguments* to the library using a group of *parameters* specific to it;

   Currently, only Hyndman's `forecast` is being considered. Under the covers, `forecast` uses the package `stats` for generic algorithms, and implements an automated hyperparameter tuning named `auto.arima`. Other R libraries are available, like `fable` by the same research group of Monash University.

   In any case, the data structure defines templates by library, not by language. Therefore, any number of libraries with different languages can be used, provided they have the same algorithm as `ctsa`;

   In this substep, study and manual programming will lead to templates and parameters that generate results equivalent to the ones of `ctsa`;

3. **Code generation**: This step is totally automatic, and from the data files, the templates and parameters, codes are generated.

   They will be compiled or simply installed in the next step (item 4);

4. **Compilation or installation**: In this step, the `C` codes are compiled, and transferred to the folder of their base libraries; the Python and R codes are just installed in the folder of their base libraries, but the installation makes sure that the setup has the expected libraries.

In more detail: the `C` code is compiled against the `ctsa` library only and some care was taken to not use other libraries. But the Python and R codes should be guaranteed to have installed all the packages or modules they need.

Any compilation failure, or absence of the needed package will generate a failure message in the compilation log.

5. **Execution**: This phase causes the results to be generated. For each execution the master database is updated, the code output (containing a statistical summary or a failure message) is redirected to a file named according to the code and execution, and a new entry is generated in the log of executions;
6. **Comparison and test**: This phase will give tools to finally test the results of the executions. This allows different versions of the same library to be compared, as well as the results of the execution of codes using two different libraries. In any case, a valid comparison will demand the same statistical model, the same *arguments*, defined in item 2a. To have a performance comparison, the same hardware should be considere. That's provided in the system description of each execution, contained in the field `sys_dec` of the table `execs`, that's defined in the item 10 of the section Tables.
7. **Additional step − More arguments**: Many of the tests in the current distribution of `ctsa` use just a single parameter for model. But obviously the testing will be much more complete if a larger number of parameters, in different ranges, are used. This module will be used to improve such quantity.

   As of now there's not a policy of adding more parameter values, but the data structure available in table `paramlims` seems to be flexible enough to accomodate any policy: only the upper and lower limits are considered.

5 Examples by hand

Worked examples of the database and software use.

6 Implementation

A shiplog reporting details of the system development, and possible deviations from the specification in the section Software design.